

Decision tree learning

Decision tree learning is a supervised learning approach used in statistics, data mining and machine learning. In this formalism, a classification or regression decision tree is used as a predictive model to draw conclusions about a set of observations.

Tree models where the target variable can take a discrete set of values are called **classification trees**; in these tree structures, leaves represent class labels and branches represent conjunctions of features that lead to those class labels. Decision trees where the target variable can take continuous values (typically real numbers) are called **regression trees**. More generally, the concept of regression tree can be extended to any kind of object equipped with pairwise dissimilarities such as categorical sequences.^[1]

Decision trees are among the most popular machine learning algorithms given their intelligibility and simplicity because they produce algorithms that are easy to interpret and visualize, even for users without a statistical background.^[2]

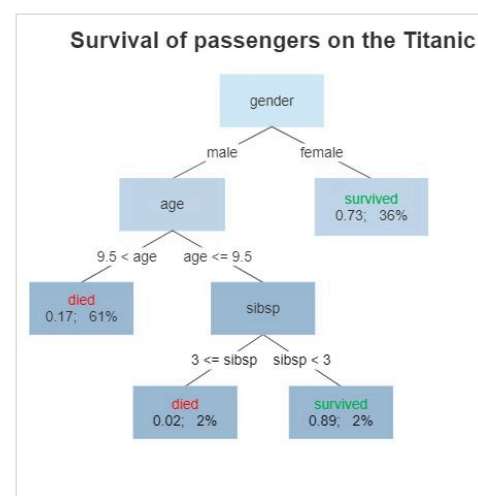
In decision analysis, a decision tree can be used to visually and explicitly represent decisions and decision making. In data mining, a decision tree describes data (but the resulting classification tree can be an input for decision making).

General

Decision tree learning is a method commonly used in data mining.^[3] The goal is to create an algorithm that predicts the value of a target variable based on several input variables.

A decision tree is a simple representation for classifying examples. For this section, assume that all of the input features have finite discrete domains, and there is a single target feature called the "classification". Each element of the domain of the classification is called a *class*. A decision tree or a classification tree is a tree in which each internal (non-leaf) node is labeled with an input feature. The arcs coming from a node labeled with an input feature are labeled with each of the possible values of the target feature or the arc leads to a subordinate decision node on a different input feature. Each leaf of the tree is labeled with a class or a probability distribution over the classes, signifying that the data set has been classified by the tree into either a specific class, or into a particular probability distribution (which, if the decision tree is well-constructed, is skewed towards certain subsets of classes).

A tree is built by splitting the source set, constituting the root node of the tree, into subsets—which constitute the successor children. The splitting is based on a set of splitting rules based on classification features.^[4] This process is repeated on each derived subset in a recursive manner called recursive partitioning. The recursion is completed when the subset at



A tree showing survival of passengers on the Titanic ("sibsp" is the number of spouses or siblings aboard). The figures under the leaves show the probability of survival and the percentage of observations in the leaf. Summarizing: Your chances of survival were good if you were (i) a female or (ii) a male at most 9.5 years old with strictly fewer than 3 siblings.

a node has all the same values of the target variable, or when splitting no longer adds value to the predictions. This process of *top-down induction of decision trees* (TDIDT)^[5] is an example of a greedy algorithm, and it is by far the most common strategy for learning decision trees from data.^[6]

In data mining, decision trees can be described also as the combination of mathematical and computational techniques to aid the description, categorization and generalization of a given set of data.

Data comes in records of the form:

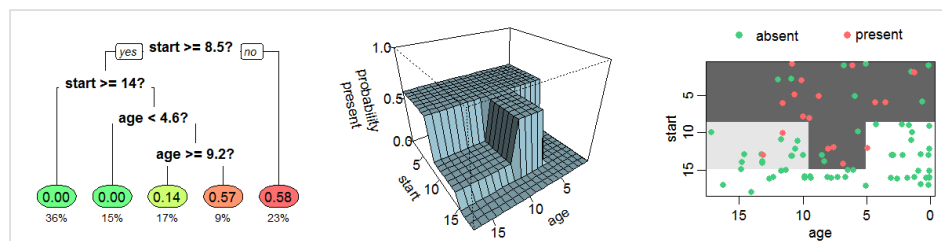
$$(\mathbf{x}, Y) = (x_1, x_2, x_3, \dots, x_k, Y)$$

The dependent variable, Y , is the target variable that we are trying to understand, classify or generalize. The vector $\mathbf{x}$ is composed of the features, x_1, x_2, x_3 etc., that are used for that task.

Decision tree types

Decision trees used in data mining are of two main types:

- **Classification tree** analysis is when the predicted outcome is the class (discrete) to which the data belongs.
- **Regression tree** analysis is when the predicted outcome can be considered a real number (e.g. the price of a house, or a patient's length of stay in a hospital).



An example tree which estimates the probability of kyphosis after spinal surgery, given the age of the patient and the vertebra at which surgery was started. The same tree is shown in three different ways. **Left** The colored leaves show the probability of kyphosis after spinal surgery, and percentage of patients in the leaf. **Middle** The tree as a perspective plot. **Right** Aerial view of the middle plot. The probability of kyphosis after surgery is higher in the darker areas. (Note: The treatment of kyphosis has advanced considerably since this rather small set of data was collected.)

The term **classification and regression tree (CART)** analysis is an umbrella term used to refer to either of the above procedures, first introduced by Breiman et al. (1984).^[7] Trees used for regression and trees used for classification have some similarities – but also some differences, such as the procedure used to determine where to split.^[7]

Some techniques, often called *ensemble* methods, construct more than one decision tree:

- **Boosted trees** Incrementally building an ensemble by training each new instance to emphasize the training instances previously mis-modeled. A typical example is AdaBoost. These can be used for regression-type and classification-type problems.^{[8][9]}
- **Committees of decision trees** (also called k-DT^[10]), an early method that used randomized decision tree algorithms to generate multiple different trees from the training data, and then combine them using majority voting to generate output.^[11]
- **Bootstrap aggregated** (or bagged) decision trees, an early ensemble method, builds multiple decision trees by repeatedly resampling training data with replacement, and voting the trees for a consensus prediction.^[12]
 - A **random forest** classifier is a specific type of bootstrap aggregating
 - **Rotation forest** – in which every decision tree is trained by first applying principal component analysis (PCA) on a random subset of the input features.^[13]

A special case of a decision tree is a decision list,^[14] which is a one-sided decision tree, so that every internal node has exactly 1 leaf node and exactly 1 internal node as a child (except for the bottommost node, whose only child is a single leaf node). While less expressive, decision lists are arguably easier to

understand than general decision trees due to their added sparsity, permit non-greedy learning methods^[15] and monotonic constraints to be imposed.^[16]

Notable decision tree algorithms include:

- ID3 (Iterative Dichotomiser 3)
- C4.5 (successor of ID3)
- CART (Classification And Regression Tree)^[7]
- OC1 (Oblique classifier 1). First method that created multivariate splits at each node.^[17]
- Chi-square automatic interaction detection (CHAID). Performs multi-level splits when computing classification trees.^{[18][19][20]}
- MARS: extends decision trees to handle numerical data better.
- Conditional Inference Trees. Statistics-based approach that uses non-parametric tests as splitting criteria, corrected for multiple testing to avoid overfitting. This approach results in unbiased predictor selection and does not require pruning.^{[21][22]}

ID3 and CART were invented independently at around the same time (between 1970 and 1980), yet follow a similar approach for learning a decision tree from training tuples.

It has also been proposed to leverage concepts of fuzzy set theory for the definition of a special version of decision tree, known as Fuzzy Decision Tree (FDT).^[23] In this type of fuzzy classification, generally, an input vector $\mathbf{x}$ is associated with multiple classes, each with a different confidence value. Boosted ensembles of FDTs have been recently investigated as well, and they have shown performances comparable to those of other very efficient fuzzy classifiers.^[24]

Metrics

Algorithms for constructing decision trees usually work top-down, by choosing a variable at each step that best splits the set of items.^[6] Different algorithms use different metrics for measuring "best". These generally measure the homogeneity of the target variable within the subsets. Some examples are given below. These metrics are applied to each candidate subset, and the resulting values are combined (e.g., averaged) to provide a measure of the quality of the split. Depending on the underlying metric, the performance of various heuristic algorithms for decision tree learning may vary significantly.^[25]

Estimate of Positive Correctness

A simple and effective metric can be used to identify the degree to which true positives outweigh false positives (see Confusion matrix). This metric, "Estimate of Positive Correctness" is defined below:

$$E_P = TP - FP$$

In this equation, the total false positives (FP) are subtracted from the total true positives (TP). The resulting number gives an estimate on how many positive examples the feature could correctly identify within the data, with higher numbers meaning that the feature could correctly classify more positive samples. Below is an example of how to use the metric when the full confusion matrix of a certain feature is given:

Feature A Confusion Matrix

Predicted Class Actual Class	Cancer	Non-cancer
Cancer	<u>8</u>	3
Non-cancer	<u>2</u>	5

Here we can see that the TP value would be 8 and the FP value would be 2 (the underlined numbers in the table). When we plug these numbers in the equation we are able to calculate the estimate: $E_p = TP - FP = 8 - 2 = 6$. This means that using the estimate on this feature would have it receive a score of 6.

However, it should be worth noting that this number is only an estimate. For example, if two features both had a FP value of 2 while one of the features had a higher TP value, that feature would be ranked higher than the other because the resulting estimate when using the equation would give a higher value. This could lead to some inaccuracies when using the metric if some features have more positive samples than others. To combat this, one could use a more powerful metric known as Sensitivity that takes into account the proportions of the values from the confusion matrix to give the actual true positive rate (TPR). The difference between these metrics is shown in the example below:

Feature A Confusion Matrix

Predicted Class Actual Class	Cancer	Non-cancer
Cancer	8	3
Non-cancer	2	5

$$E_p = TP - FP = 8 - 2 = 6$$

$$TPR = TP / (TP + FN) = 8 / (8 + 3) \approx 0.73$$

Feature B Confusion Matrix

Predicted Class Actual Class	Cancer	Non-cancer
Cancer	6	2
Non-cancer	2	8

$$E_p = TP - FP = 6 - 2 = 4$$

$$TPR = TP / (TP + FN) = 6 / (6 + 2) = 0.75$$

In this example, Feature A had an estimate of 6 and a TPR of approximately 0.73 while Feature B had an estimate of 4 and a TPR of 0.75. This shows that although the positive estimate for some feature may be higher, the more accurate TPR value for that feature may be lower when compared to other features that have a lower positive estimate. Depending on the situation and knowledge of the data and decision trees, one may opt to use the positive estimate for a quick and easy solution to their problem. On the other hand, a more experienced user would most likely prefer to use the TPR value to rank the features because it takes into account the proportions of the data and all the samples that should have been classified as positive.

Gini impurity

Gini impurity, **Gini's diversity index**,^[26] or **Gini-Simpson Index** in biodiversity research, is used by the CART (classification and regression tree) algorithm for classification trees. Gini impurity is the probability that a randomly chosen element of a set would be mislabeled if it were labeled randomly and independently according to the distribution of labels in the set. It reaches its minimum (zero) when all cases in the node fall into a single target category.

For a set of items with J classes and relative frequencies p_i , $i \in \{1, 2, \dots, J\}$, the probability of choosing an item with label i is p_i , and the probability of miscategorizing that item is $\sum_{k \neq i} p_k = 1 - p_i$. The Gini

impurity is computed by summing pairwise products of these probabilities for each class label:

$$I_G(p) = \sum_{i=1}^J \left(p_i \sum_{k \neq i} p_k \right) = \sum_{i=1}^J p_i (1 - p_i) = \sum_{i=1}^J (p_i - p_i^2) = \sum_{i=1}^J p_i - \sum_{i=1}^J p_i^2 = 1 - \sum_{i=1}^J p_i^2.$$

The Gini impurity is also an information theoretic measure and corresponds to Tsallis Entropy with deformation coefficient $q = 2$, which in physics is associated with the lack of information in out-of-equilibrium, non-extensive, dissipative and quantum systems. For the limit $q \rightarrow 1$ one recovers the usual Boltzmann-Gibbs or Shannon entropy. In this sense, the Gini impurity is nothing but a variation of the usual entropy measure for decision trees.

Information gain

Used by the ID3, C4.5 and C5.0 tree-generation algorithms. Information gain is based on the concept of entropy and information content from information theory.

Entropy is defined as below

$$H(T) = I_E(p_1, p_2, \dots, p_J) = - \sum_{i=1}^J p_i \log_2 p_i$$

where $p_1, p_2, \dots$ are fractions that add up to 1 and represent the percentage of each class present in the child node that results from a split in the tree.^[27]

$$\begin{aligned} \overbrace{IG(T, a)}^{\text{information gain}} &= \overbrace{H(T)}^{\text{entropy (parent)}} - \overbrace{H(T | a)}^{\text{sum of entropies (children)}} \\ &= - \sum_{i=1}^J p_i \log_2 p_i - \sum_{i=1}^J - \Pr(i | a) \log_2 \Pr(i | a) \end{aligned}$$

Averaging over the possible values of A ,

$$\begin{aligned} \overbrace{E_A(IG(T, a))}^{\text{expected information gain}} &= \overbrace{I(T; A)}^{\text{mutual information between } T \text{ and } A} = \overbrace{H(T)}^{\text{entropy (parent)}} - \overbrace{H(T | A)}^{\text{weighted sum of entropies (children)}} \\ &= - \sum_{i=1}^J p_i \log_2 p_i - \sum_a p(a) \sum_{i=1}^J - \Pr(i | a) \log_2 \Pr(i | a) \end{aligned}$$

Where weighted sum of entropies is given by,

$$H(T | A) = \sum_a p(a) \sum_{i=1}^J - \Pr(i | a) \log_2 \Pr(i | a)$$

That is, the expected information gain is the mutual information, meaning that on average, the reduction in the entropy of T is the mutual information.

Information gain is used to decide which feature to split on at each step in building the tree. Simplicity is best, so we want to keep our tree small. To do so, at each step we should choose the split that results in the most consistent child nodes. A commonly used measure of consistency is called information which is measured in bits. For each node of the tree, the information value "represents the expected amount of information that would be needed to specify whether a new instance should be classified yes or no, given that the example reached that node".^[27]

Consider an example data set with four attributes: *outlook* (sunny, overcast, rainy), *temperature* (hot, mild, cool), *humidity* (high, normal), and *windy* (true, false), with a binary (yes or no) target variable, *play*, and 14 data points. To construct a decision tree on this data, we need to compare the information gain of each of four trees, each split on one of the four features. The split with the highest information gain will be taken as the first split and the process will continue until all children nodes each have consistent data, or until the information gain is 0.

To find the information gain of the split using *windy*, we must first calculate the information in the data before the split. The original data contained nine yes's and five no's.

$$I_E([9, 5]) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

The split using the feature *windy* results in two children nodes, one for a *windy* value of true and one for a *windy* value of false. In this data set, there are six data points with a true *windy* value, three of which have a *play* (where *play* is the target variable) value of yes and three with a *play* value of no. The eight remaining data points with a *windy* value of false contain two no's and six yes's. The information of the *windy*=true node is calculated using the entropy equation above. Since there is an equal number of yes's and no's in this node, we have

$$I_E([3, 3]) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$$

For the node where *windy*=false there were eight data points, six yes's and two no's. Thus we have

$$I_E([6, 2]) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.81$$

To find the information of the split, we take the weighted average of these two numbers based on how many observations fell into which node.

$$I_E([3, 3], [6, 2]) = I_E(\text{windy or not}) = \frac{6}{14} \cdot 1 + \frac{8}{14} \cdot 0.81 = 0.89$$

Now we can calculate the information gain achieved by splitting on the *windy* feature.

$$IG(\text{windy}) = I_E([9, 5]) - I_E([3, 3], [6, 2]) = 0.94 - 0.89 = 0.05$$

To build the tree, the information gain of each possible first split would need to be calculated. The best first split is the one that provides the most information gain. This process is repeated for each impure node until the tree is complete. This example is adapted from the example appearing in Witten et al.^[27]

Information gain is also known as Shannon index in bio diversity research.

Variance reduction

Introduced in CART,^[7] variance reduction is often employed in cases where the target variable is continuous (regression tree), meaning that use of many other metrics would first require discretization before being applied. The variance reduction of a node N is defined as the total reduction of the variance of the target variable Y due to the split at this node:

$$I_V(N) = \frac{1}{|S|^2} \sum_{i \in S} \sum_{j \in S} \frac{1}{2} (y_i - y_j)^2 - \left(\frac{|S_t|}{|S|} \frac{1}{|S_t|^2} \sum_{i \in S_t} \sum_{j \in S_t} \frac{1}{2} (y_i - y_j)^2 + \frac{|S_f|}{|S|} \frac{1}{|S_f|^2} \sum_{i \in S_f} \sum_{j \in S_f} \frac{1}{2} (y_i - y_j)^2 \right)$$

where $\mathcal{S}$, $\mathcal{S}_t$, and $\mathcal{S}_f$ are the set of presplit sample indices, set of sample indices for which the split test is true, and set of sample indices for which the split test is false, respectively. Each of the above summands are indeed variance estimates, though, written in a form without directly referring to the mean.

By replacing $(y_i - y_j)^2$ in the formula above with the dissimilarity d_{ij} between two objects i and j , the variance reduction criterion applies to any kind of object for which pairwise dissimilarities can be computed.^[1]

Measure of "goodness"

Used by CART in 1984,^[28] the measure of "goodness" is a function that seeks to optimize the balance of a candidate split's capacity to create pure children with its capacity to create equally-sized children. This process is repeated for each impure node until the tree is complete. The function $\varphi(\mathbf{s} \mid \mathbf{t})$, where $\mathbf{s}$ is a candidate split at node $\mathbf{t}$, is defined as below

$$\varphi(\mathbf{s} \mid \mathbf{t}) = 2P_L P_R \sum_{j=1}^{\text{class count}} |P(j \mid \mathbf{t}_L) - P(j \mid \mathbf{t}_R)|$$

where $\mathbf{t}_L$ and $\mathbf{t}_R$ are the left and right children of node $\mathbf{t}$ using split $\mathbf{s}$, respectively; P_L and P_R are the proportions of records in $\mathbf{t}$ in $\mathbf{t}_L$ and $\mathbf{t}_R$, respectively; and $P(j \mid \mathbf{t}_L)$ and $P(j \mid \mathbf{t}_R)$ are the proportions of class j records in $\mathbf{t}_L$ and $\mathbf{t}_R$, respectively.

Consider an example data set with three attributes: *savings*(low, medium, high), *assets*(low, medium, high), *income*(numerical value), and a binary target variable *credit risk*(good, bad) and 8 data points.^[28] The full data is presented in the table below. To start a decision tree, we will calculate the maximum value of $\varphi(\mathbf{s} \mid \mathbf{t})$ using each feature to find which one will split the root node. This process will continue until all children are pure or all $\varphi(\mathbf{s} \mid \mathbf{t})$ values are below a set threshold.

Customer	Savings	Assets	Income (\$1000s)	Credit risk
1	Medium	High	75	Good
2	Low	Low	50	Bad
3	High	Medium	25	Bad
4	Medium	Medium	50	Good
5	Low	Medium	100	Good
6	High	High	25	Good
7	Low	Low	25	Bad
8	Medium	Medium	75	Good

To find $\varphi(\mathbf{s} \mid \mathbf{t})$ of the feature *savings*, we need to note the quantity of each value. The original data contained three low's, three medium's, and two high's. Out of the low's, one had a good *credit risk* while out of the medium's and high's, 4 had a good *credit risk*. Assume a candidate split $\mathbf{s}$ such that records with a low *savings* will be put in the left child and all other records will be put into the right child.

$$\varphi(\mathbf{s} \mid \text{root}) = 2 \cdot \frac{3}{8} \cdot \frac{5}{8} \cdot \left(\left| \left(\frac{1}{3} - \frac{4}{5} \right) \right| + \left| \left(\frac{2}{3} - \frac{1}{5} \right) \right| \right) = 0.44$$

To build the tree, the "goodness" of all candidate splits for the root node need to be calculated. The candidate with the maximum value will split the root node, and the process will continue for each impure node until the tree is complete.

Compared to other metrics such as information gain, the measure of "goodness" will attempt to create a more balanced tree, leading to more-consistent decision time. However, it sacrifices some priority for creating pure children which can lead to additional splits that are not present with other metrics.

Uses

Advantages

Amongst other data mining methods, decision trees have various advantages:

- **Simple to understand and interpret.** People are able to understand decision tree models after a brief explanation. Trees can also be displayed graphically in a way that is easy for non-experts to interpret.^[29]
- **Able to handle both numerical and categorical data.**^[29] Other techniques are usually specialized in analyzing datasets that have only one type of variable. (For example, relation rules can be used only with nominal variables while neural networks can be used only with numerical variables or categoricals converted to 0-1 values.) Early decision trees were only capable of handling categorical variables, but more recent versions, such as C4.5, do not have this limitation.^[3]
- **Requires little data preparation.** Other techniques often require data normalization. Since trees can handle qualitative predictors, there is no need to create dummy variables.^[29]
- **Uses a white box or open-box**^[3] **model.** If a given situation is observable in a model the explanation for the condition is easily explained by Boolean logic. By contrast, in a black box model, the explanation for the results is typically difficult to understand, for example with an artificial neural network.
- **Possible to validate a model using statistical tests.** That makes it possible to account for the reliability of the model.
- Non-parametric approach that makes no assumptions of the training data or prediction residuals; e.g., no distributional, independence, or constant variance assumptions
- **Performs well with large datasets.** Large amounts of data can be analyzed using standard computing resources in reasonable time.
- **Accuracy with flexible modeling.** These methods may be applied to healthcare research with increased accuracy.^[30]
- **Mirrors human decision making more closely than other approaches.**^[29] This could be useful when modeling human decisions/behavior.
- **Robust against co-linearity, particularly boosting.**
- **In built feature selection.** Additional irrelevant feature will be less used so that they can be removed on subsequent runs. The hierarchy of attributes in a decision tree reflects the importance of attributes.^[31] It means that the features on top are the most informative.^[32]
- **Decision trees can approximate any Boolean function e.g. XOR.**^[33]

Limitations

- Trees can be very non-robust. A small change in the training data can result in a large change in the tree and consequently the final predictions.^[29]
- The problem of learning an optimal decision tree is known to be NP-complete under several aspects of optimality and even for simple concepts.^{[34][35]} Consequently, practical decision-tree learning algorithms are based on heuristics such as the greedy algorithm where locally optimal decisions are made at each node. Such algorithms cannot guarantee to return the globally optimal decision tree. To reduce the greedy effect of local optimality, some methods such as the dual information distance (DID) tree were proposed.^[36]
- Decision-tree learners can create over-complex trees that do not generalize well from the training data. (This is known as overfitting.^[37]) Mechanisms such as pruning are necessary to avoid this